

5 Things That Make Claude Code a System, Not a Chatbot

A folder, a memory, a rules-only file, a model table, and never trusting done. Each one explained in plain English, plus one prompt that sets all five up for you.

Built by [@brycenwood.ai](#) · September 2026

WHY THESE FIVE

A chat forgets. A system remembers, follows rules, spends carefully, and proves what it did. These are the five moves that make the difference, in the order I would do them.

ONE

Work out of a folder, not a chat

A chat forgets everything the second you close it. A folder does not. When Claude Code runs inside a folder, every file in it is something it can go open and read on its own, so you stop re-explaining your business every morning.

How: make a folder for the project. Open the terminal inside it and type `c l a u d e`. The first thing you say is *run init*. It writes a file called `CLAUDE.md`. That file is read at the start of every session. It is where you tell it who you are, what this is, and how you want it to talk to you.

Give it a memory

Memory is not magic. It is a folder plus one rule. Mine has over five hundred files in it, one fact per file, and one index file that lists them in a line each. Tomorrow's session reads the index first and knows what yesterday learned.

How: add a folder called memory with a MEMORY.md index inside it. Then put one rule in CLAUDE.md: *Before a session ends, write anything durable you learned as one file in memory/ and add a one-line pointer to MEMORY.md. Read MEMORY.md at the start of every session.* Then actually say *write down what you learned* before you close. The rule is the habit. The folder is just where it lands.

THREE

Rules only in the main file

CLAUDE.md loads on every single request. Every line in it costs attention on everything it does. So the more you help it by stuffing that file, the worse it gets. Mine is rules only, with a hard line cap.

How: add this rule to CLAUDE.md: *This file is rules only and stays under 150 lines. Anything only needed sometimes goes in its own file and this file points to it.* Then ask it to check the current file against that rule and move anything that is not a rule. The wrap-shop details go in a wrap-shop file. The main file just says where that file is.

FOUR

One table: which model for which job

Claude comes in sizes. The biggest brain costs about seven times more per call than the smallest. If you let it guess, it uses the biggest brain to rename a file. One small table fixes that forever.

How: add a section to CLAUDE.md called *Which model for which job*. Sorting, extracting, summarizing and routing go to the smallest model. Drafting, analysis and generating go to the middle one. Reviewing work and design decisions go to the biggest. Then the rule that makes it stick: *When you spawn an agent or write a script that calls the API, look the job up in this table. Never guess.*

FIVE

Never trust done

Mine once told me a message queue had sent zero. Meanwhile 162 customers had already been answered. Both were true. It was reading its own notes instead of the actual log. Confident is not the same as true.

How: two rules in CLAUDE.md: *Nothing is done until you show me the actual output or log, never your summary of it.* and *A command that exits clean with no output is a failure, not a pass.* Then the one that matters most, a **hook**: a tiny script that runs before every command it tries and blocks anything that deletes files or spends money. A rule is something it tries to follow. A hook is something it cannot get past. Ask it to build the hook and explain what it blocks before it saves it.

THE PROMPT

One paste sets up all five

Make a folder for the project. Open the terminal inside it and type `claude`. Paste this and answer its questions one at a time. It shows you everything it built at the end.

PASTE THIS INTO CLAUDE CODE

Set up this folder as a system, not a chat. Do these five things, and ask me one question at a time when you need something from me.

1. FOLDER. If there is no CLAUDE.md here, create one. Ask me what this project is, who I am, and how I want you to talk to me. Keep it short.

2. MEMORY. Create a folder called memory with a MEMORY.md index inside. Add this rule to CLAUDE.md: "Before a session ends, write anything durable you learned (a preference, a gotcha, a decision and why) as one file per fact in memory/, and add a one-line pointer to MEMORY.md. Read MEMORY.md at the start of every session."

3. RULES ONLY. Add this rule to CLAUDE.md: "This file is rules only and stays under 150 lines. Anything only needed sometimes goes in its own file under docs/ and this file points to it." Then check the current file against that and move anything that is not a rule.

4. MODEL TABLE. Add a section to CLAUDE.md called Which model for which job, with a table: classify, extract, summarize and route go to the smallest model; drafting, analysis and generating go to the middle model; reviewing work and design decisions go to the largest model. Add the rule: "When you spawn an agent or write a script that calls

the API, look the job up in this table. Never guess."

5. TRUST. Add these rules to CLAUDE.md: "Nothing is done until you show me the actual output or log, never your summary of it." and "A command that exits clean with no output is a failure, not a pass." Then set up a PreToolUse hook that runs before every Bash command and blocks anything that deletes files, force-pushes, or spends money. Explain to me in plain English what it blocks before you save it.

When all five are done, show me the CLAUDE.md, the memory folder, and the hook, and tell me the one thing you would add next.

That is the whole system.

Zero ad spend. Zero coding experience. The other playbooks
live at brycenwood.com/playbook, one download each.

Built by Brycen Wood · [@brycenwood.ai](https://twitter.com/brycenwood.ai) · brycenwood.com